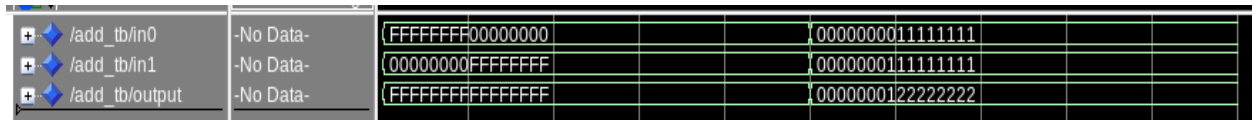


# Lab2 report

This lab contains the major components of a CPU based on ARM. The CPU consists of four main parts in total. Thus, this report will be divided into four parts, in which I will illustrate the functionality of my code and give out the simulation wave form to demonstrate what is happening.

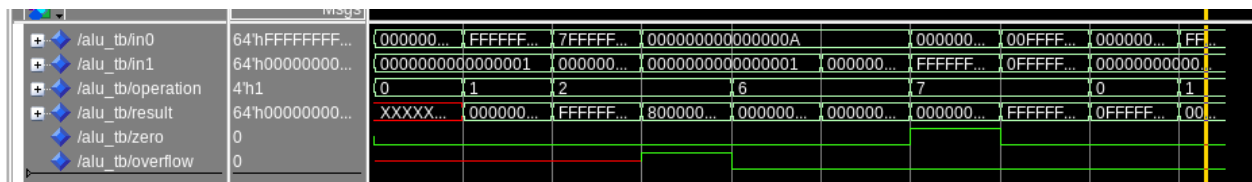
## 1. ALU part

The following figure shows my simulation of the ADD components.



Two waveform is performed. Each waveform shows the output is doing “add” action for input in0 and in1.

Then comes the waveform of the main part of ALU component.



The input 'in0' and 'in1' are two 64-bit-operands that need to be calculated. Another input 'operation' controls which action that the ALU needs to do with two operands. Zeros detects whether the output is zero and the overflow shows that whether the signed calculation turns into the wrong result.

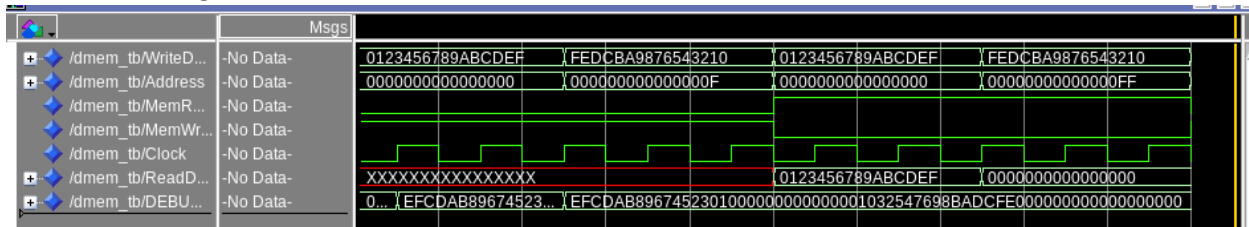
From the figure above, we can see that the output is 50 ns later than input. When the input is X"000000000000000F" and X"0000000000000001" and operation is 'and', the output shows nothing. When 50 ns past, the output begins to illustrate what's going on before. That means the ALU has an obvious buffer in it.

Another interesting waveform is that when the third input starts, the ALU is performing an 'add' operation, accompanied by an overflow. So, in the fourth period, the output overflow gets a '1' instead of 'X' before.

The similar thing occurs in the sixth period, where the ALU is doing subtract for two same inputs. That cause a zero outcome. Thus, the output zero gets a '1' in the seventh period.

## 2. DMEM part

The function of DMEM component is to store the data given out by register and ALU. The waveform of its simulation is given as:



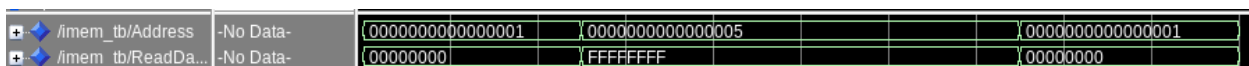
The first two period is doing 'write' options to the DMEM and the last two period is doing 'read' action.

The first thing that I notice is DEBUG\_MEM\_CONTENTS shows its contents in an opposite-pairs digit. The reason for it is the order of store isn't fit for the order that display in DEBUG\_MEM\_CONTENTS.

The effectiveness of the storage has been shown, because from the debug bit we can see that the input data has been stored in the corresponding location. And the last two period, when doing 'read' action, also shows the effectiveness of 'write' and the initialization. Especially for the last period, we are trying to read an unwritten location, which has been initialized as 0. The output of it confirm what we had done.

## 3. IMEM part

The IMEM part is simpler than the DMEM part. Also they function similarly, although one is for storing data and the other is for storing commands. The simulation waveform is shown below:



I only initialize the 8 bytes of IMEM, where the first four stores X"00000000" and the later four stores X"FFFFFFF". The PC read the instructions in a group of four bytes. Therefore, from the waveform given, output of 'ReadData' is firstly '00000000' and then 'FFFFFFF'.

## 4. Registers part

The register is still pretty the same as DMEM part. According to the experimental requirements, I initialized X9, X10, X11, X12, X13, X14, X15 and XZR bits, namely X31. Also put X9, X10, X11, X12 into the DEBUG\_TMP\_REGS. In the testbench, I tried to store X19, X20 in the simulation.

